Problem Situation - Evidence 2

Ethiel Favila Alvarado
Programming of Data Structures and Fundamental Algorithms
A00844789@tec.mx

I. WHAT IS THE PROBLEM, AND WHY IS IT RELEVANT?

The problem is that we need to identify, organize and analyze failed authentication attempts and unauthorized access requests logged in the system. It is important we analyze them properly as it is a security matter. The unsorted security logs contained a lot of data which, without processing, is difficult to detect breaches or malicious IP addresses. To organize them better increased the chances of bettering security.

II. HOW DID YOU SOLVE THE PROBLEM

To solve the issue of the authentication reports being unordered and inaccessible we focused on being able to safely store them in a way we could access different parts of the logs (timestamps, text, IP, etc.) and easily sort and filter them. First, we read the logs from bitacora.txt and store them in keeper on a doubly linked list. Then, we use the class Sorter that first orders the list by IP address and then orders them by time. After, we ask for a start and end IP by using Searcher, who then extracts every record in the range. Since the list is already sorted all matching records are close, and IP's are already in time order. Finally, the output is both printed and stored in output.txt.

We have six classes including main: Entry, Keeper, main, Searcher, Sorter and Stack. Entry is a struct part of Entry which holds one log record: month, day, time, IP, and message Then we have the node struct and Keeper class. The node is a struct that holds a previous and next pointer, as well as one Entry object, as so we construct a doubly linked list. We have sorter that calls the iterative quicksort. And then StackNode (part of the Stack class) that builds a singly linked list and replaces recursion in the quicksort. Finally we have the Searcher class that makes the final search,

III. DATA STRUCTURES AND ALGORITHMS

A. Which data structures and algorithms were used? How were they implemented, and why were they selected?

- *Double linked list*- Each log line is put into Entry (month, day, time, IP, message and timestamp). Then each entry has a node with next and prev pointers. We can keep adding nodes indefinitely. It was used this way because we didn't know how many records, we had originally, so the list grows each time instead of assigning each part to an already existing spot.
- *Stack on the single linked list*- The top stack is the head of the singly linked list. It has the functions: push and pop at the head, and both element has to pointers (low, high) which marks which part of the list still needs sorting. It was used this way because it replaces recursion in quicksort, we avoid running the quicksort too many times.
- *Iterative quicksort*- Records are first ordered by IP the date and time. The IP part is done by turning each part (like a.b.c) into a base 1000 number, that way we could compare actual decimal numbers instead of the a.b.c.d structure. It was used this way because quicksort is an efficient sort, but moreover, we don't need to copy the records (which are a lot) a second time like merge sort would.
- *Range search*- We look for the start and end IP's before searching, if start>end then we just switch places (this is a safety in case they give us the order wrong). Then we skip until we find the start IQ and start collecting matches, but as soon as we pass the end IP we stop. We used this way because the linked list has no random access, so binary search doesn't really work. The linear search, although basic, works in this case since things are already sorted.

B. Is a linked list an appropriate data structure for this particular problem? Explain why or why not.

In our case a linked list works because we don't know the number of records. Since we don't know how many records we have, using pushBack helps grow the list dynamically, without ever having to resize or copy data. Furthermore, we can easily output the results after sorting. When Searcher finds the first and last record we just store which ones they are and print and store whatever is between, without having to copy anything (and thus avoiding wasted memory space). Plus, insertions and deletions are only O(1).

However, linked list have deficiencies in this case too. First, we cannot use binary search because we cannot jump to the middle of an element. We have to compensate by walking to the head one node at the time which costs more time. Plus, we use more memory as beside the entry we have two extra pointers per entry.

C. What are the advantages and disadvantages of the data structures and algorithms used?

TABLE I
ADVANTAGES AND DISADVANATGES

Algorithm	**Advantages**	**Disadvantages**
Doubly linked list	It grows dynamically, so we don't need to know the amount of records form the beginning. Records can be swapped without messing with other records. Prev allows us to easily use quicksort.	We don't have random access, so we can't use binary search. Besides it uses extra memory as we have two pointers per node.
Stack	We can add as many records as we want without it overflowing.	Each element need an extra next pointer, which normally wouldn't happen with an array.
Iterative quicksort	It sorts in place,	It has a worse

	data is only swapped in nodes. The quicksort partition only need sequential access (not random access) which fits the linked list. Its fast, besides the iterative part helps with it not overflowing.	case scenario of O(n^2) which depends on the order of the input. Merge sort offers the guarantee of O(n log n) which doesn't happen here I. which the worst case might be really worse..
Range search	It is simple, so its difficult to mess it. Since the information is already sorted it stops early. We obtain a first and last pointer, without complications.	The best case is the same as the worst case O(1), other search like binary search has a better best case scenario. In our case it works because everything is already ordered, but in general this is a disadvantage.

D. What are their best- and worst-case computational complexities, and what determines those cases?

TABLE II
BEST CASE AND WORST-CASE COMPLEXITIES OF ALL OPERATIONS

Algorithm	**Operation**	**Best case**	**Worst case**	**What determines the case?**
Doubly linked list	pushback	O(1)	O(1)	Always the same thanks to the tail pointer
	Read	O(n)	O(n)	Each line n is read and added
	destructor	O(n)	O(n)	Every node must be freed
Stack	Push	O(1)	O(1)	Always works with the top part
	Pop	O(1)	O(1)	Always works with the top part
	isEmpty	O(1)	O(1)	Always works with the top part
Iterative quicksort	Quicksort	O(n log n)	O(n^2)	The best comes from the pivot landing in the middle every time and the list is split in half perfectly. Worst case is the pivot is always the smallest/highest number.
Range search	General search	O(1)	O(n)	Best case it finds it in the first file, worst case it's the last

IV. IMPROVEMENTS

The code works and efficiently manages all the records, however, we could make some improvements. First, we could try to is merge sort instead of quicksort. Merge sort can still work on linked lists but guarantees an O(n log n) time, regardless of the pivot. We could also handle more cases. In bitacora.txt, all logs are formatted the same, however we don't account for corrupted lines. We don't consider this when parsing.

And interesting possible feature would be repeated searches, allowing the user to run several queries without having to restart the program. Besides, the whole point of the organization is to detect failed attempt, so it would be nice to flag Ips that have many failed attempts in short times (which is easy because we organize the Ips by time). Besides that, a filter not just by IP but by attack time would be interesting.

V. DIAGRAMS

DIAGRAM I
FLOW CHART OF THE PROGRAM IN GENERAL

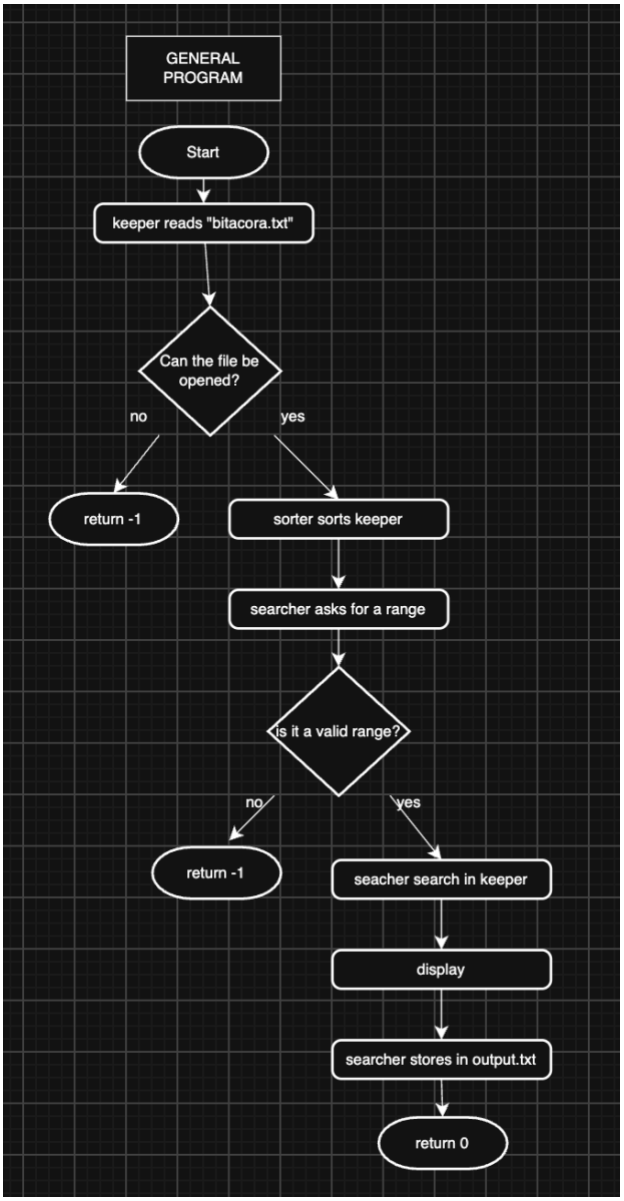


DIAGRAM II
HOW THE LINKED LIST IS CREATED

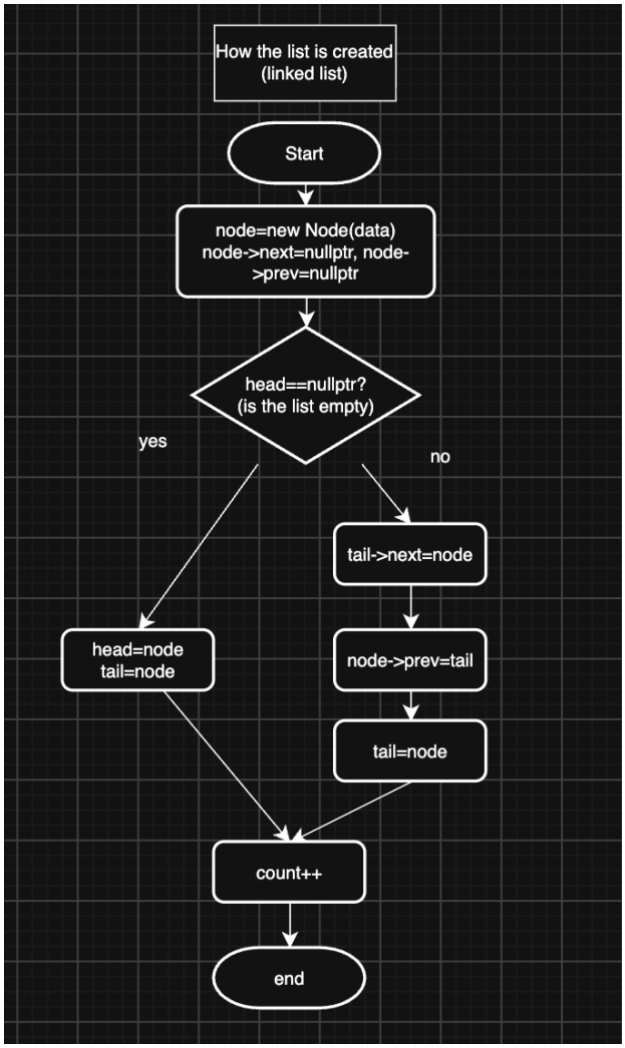


DIAGRAM III
HOW THE LINKED LIST IS SORTED

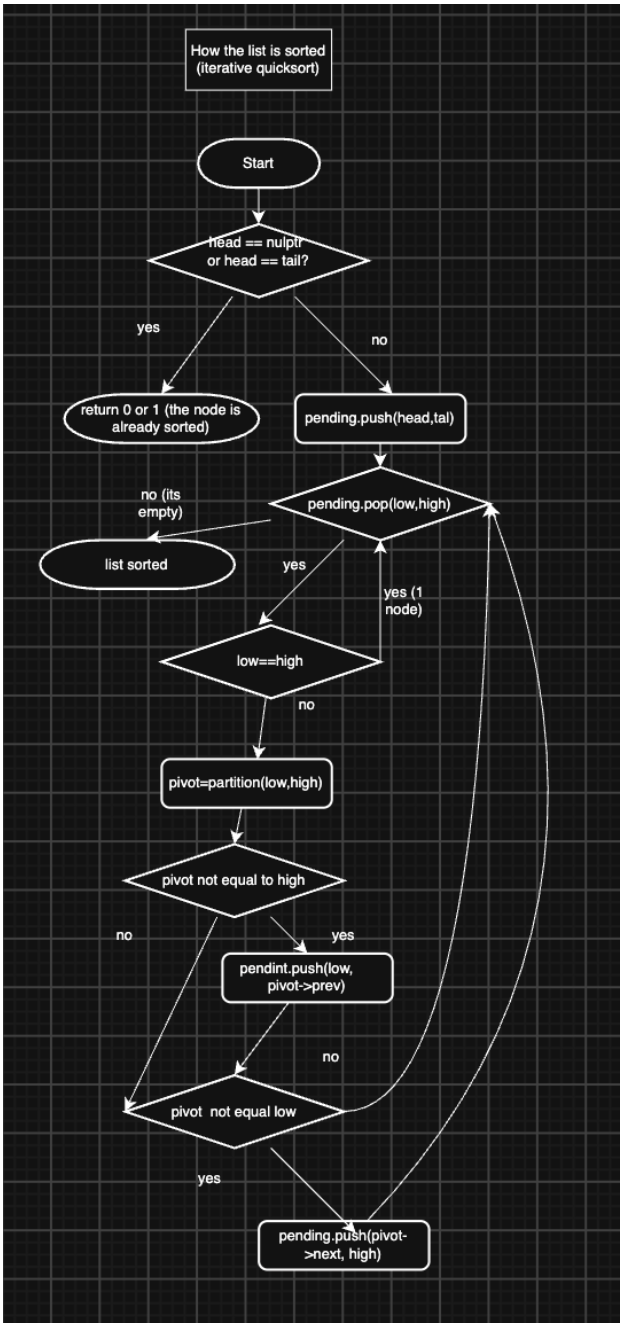
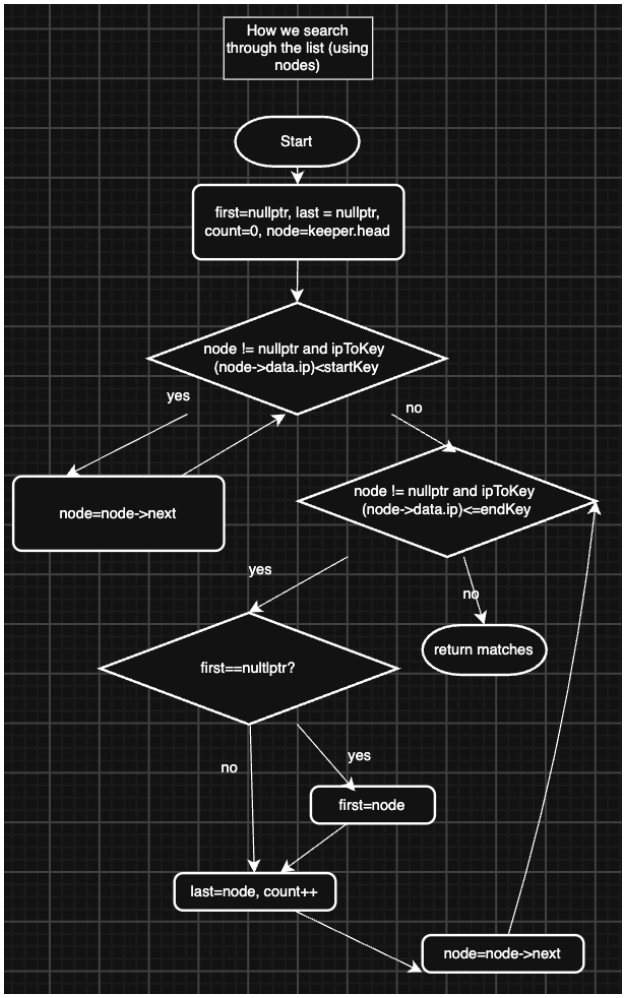


DIAGRAM IV
SEARCH METHOD



VI. REFERENCES

[1] OWASP Foundation, “Logging Cheat Sheet,” OWASP Cheat Sheet Series. [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/Logging_Cheat_Sheet.html [Accessed: Oct. 20, 2026].

[2] OpenDSA, “13.11. Quicksort,” OpenDSA Data Structures and Algorithms Modules. [Online]. Available: https://opendsa-server.cs.vt.edu/ODSA/Books/pubbook/odsa-all-0013bb43-a12f-4ab3-9b86-73b317f1e37e/summer-i-2018/Pub_Sum2018_All_ODSA/html/Quicksort.html [Accessed: Oct. 20, 2026].